

Lightweight RPC for Kotlin Multiplatform

by

Aleksandr Stupnikov

Master Thesis in Computer Science and Software Engineering

Submission: May 19, 2026

Supervisor: Dr. A. Podkopaev

Statutory Declaration

Family Name, Given/First Name	Stupnikov, Aleksandr
Matriculation number	30008334
Kind of thesis submitted	Master Thesis

English: Declaration of Authorship

I hereby declare that the thesis submitted was created and written solely by myself without any external support. Any sources, direct or indirect, are marked as such. I am aware of the fact that the contents of the thesis in digital form may be revised with regard to usage of unauthorized aid as well as whether the whole or parts of it may be identified as plagiarism. I do agree my work to be entered into a database for it to be compared with existing sources, where it will remain in order to enable further comparisons with future theses. This does not grant any rights of reproduction and usage, however.

This document was neither presented to any other examination board nor has it been published.

German: Erklärung der Autorenschaft (Urheberschaft)

Ich erkläre hiermit, dass die vorliegende Arbeit ohne fremde Hilfe ausschließlich von mir erstellt und geschrieben worden ist. Jedwede verwendeten Quellen, direkter oder indirekter Art, sind als solche kenntlich gemacht worden. Mir ist die Tatsache bewusst, dass der Inhalt der Thesis in digitaler Form geprüft werden kann im Hinblick darauf, ob es sich ganz oder in Teilen um ein Plagiat handelt. Ich bin damit einverstanden, dass meine Arbeit in einer Datenbank eingegeben werden kann, um mit bereits bestehenden Quellen verglichen zu werden und dort auch verbleibt, um mit zukünftigen Arbeiten verglichen werden zu können. Dies berechtigt jedoch nicht zur Verwendung oder Vervielfältigung.

Diese Arbeit wurde noch keiner anderen Prüfungsbehörde vorgelegt noch wurde sie bisher veröffentlicht.

Disclosure of AI Use

During the preparation of this master's thesis, artificial intelligence tools were utilized as supplementary aids in the writing process. Specifically, AI was used to generate illustrative examples and to assist in drafting descriptions for certain sections of the research and methodology to enhance clarity. All AI-assisted content was thoroughly reviewed, critically evaluated, and significantly edited by the author to ensure accuracy, originality, and alignment with academic standards.

.....
Date, Signature

Abstract

A large subset of modern software for Kotlin and Kotlin Multiplatform (KMP) uses network interactions extensively. Backend microservices, client-server applications and distributed systems are typical examples. Many such projects follow *shared-codebase* model in which client and server live in a single Kotlin project and share data types and parts of business logic. Existing RPC frameworks for Kotlin like gRPC and Kotlin require developer to describe remote functions in a separate service interface. This interface plays the role of a contract between client and server, which is necessary when client and server are developed independently. In shared-codebase setting this contract becomes redundant, because client and server already share the codebase and signatures of remote functions can be the contract on their own. The interface declaration then turns into *boilerplate code* that exists only because the chosen framework requires it.

In this thesis we developed a new RPC framework for shared-codebase Kotlin projects. We called this new framework *Kotlin Remote* [25]. Kotlin Remote uses Kotlin's experimental *context parameters* feature to express remote calls without a service interface declaration. The context parameter of a remote function carries information about the target machine, which makes the function callable on that machine, and at the same time appears in the function signature, which makes the remoteness of the function explicit at the declaration level.

Kotlin Remote is intentionally scoped to the shared-codebase case. It does not try to compete with general purpose RPC frameworks like gRPC or Kotlin RPC outside this scope. In particular, it does not support cross-language communication and is not meant for projects in which client and server are developed independently in separate codebases. In those scenarios the service interface required by gRPC or Kotlin RPC is not redundant, so the trade made by Kotlin Remote does not pay off. Inside its target scope, however, Kotlin Remote removes the redundant interface declaration and leads to substantial reduction of boilerplate compared to existing alternatives, while keeping remote calls explicit for the developer. Shared-codebase model is very common in Kotlin and KMP environment, so we believe Kotlin Remote will be useful despite its narrow scope.

Contents

1	Introduction	1
1.1	Goal and objectives	2
2	Background and Related work	3
2.1	RPC	3
2.2	Distributed Objects	4
2.3	REST	4
2.4	gRPC	5
2.5	Kotlin RPC	5
2.6	Coeffects	6
2.7	Multi-tier programming languages	8
2.8	Kotlin Ecosystem	8
2.8.1	Kotlin Coroutines	8
2.8.2	Kotlin Multiplatform	9
2.8.3	kotlinx.serialization	9
2.8.4	Ktor	9
2.8.5	Kotlin context parameters	9
3	Kotlin Remote Implementation	10
3.1	Module structure	10
3.2	Remote functions	11
3.2.1	Client	11
3.2.2	Server	14
3.2.3	Unified client and server bodies via <code>RemoteContext</code>	15
3.2.4	Remote context relations	16
3.3	Remote functions code generation	17
3.3.1	Remote function body generation	18
3.3.2	Callable map generation	18
3.4	Exception handling	19
3.5	Ktor integration	20
3.6	Remote classes	21
3.6.1	Remote classes code generation	22
3.7	Garbage collection	23
4	Evaluation of Kotlin Remote	24
4.1	Example applications	25
4.1.1	Todo application	25
4.1.2	Rooms chat	26
4.1.3	Social platform	27
4.1.4	CMS	28
4.2	Boilerplate comparison with Kotlin RPC	29
4.2.1	Per-function boilerplate overhead	31
4.3	Performance comparison with Kotlin RPC	32
4.3.1	Per-call latency	32
4.3.2	Local dispatch overhead	33
4.3.3	Distributed garbage collection	33
4.3.4	Per-function bytecode footprint	34
4.3.5	Discussion	34

5	Limitations and Future Work	34
5.1	Limitations	35
5.1.1	Narrow scope of application	35
5.1.2	Dependency on experimental Kotlin features	35
5.2	Future work	35
5.2.1	Code slicing	35
5.2.2	Remote lambdas	36
5.2.3	Bidirectional streaming	36
6	Conclusions	36

1 Introduction

Nowadays, the Kotlin programming language is used frequently for developing software that relies heavily on network interactions. Such software includes backend microservices, client-server applications and distributed systems. Many of these projects follow the *shared-codebase* model, in which client and server live in a single Kotlin project and share data types and parts of the business logic. By a shared-codebase project we mean a project where all the parts that communicate over the network are developed together, compiled together and released together. This is the setting this thesis is concerned with. The opposite setting, in which client and server are developed independently, often in different languages by different teams, is out of scope.

The codebase of a shared-codebase application can usually be roughly separated into two parts: the part responsible for the actual business logic, and the part responsible for network interactions. While the former part differs significantly from program to program, the network interactions part is usually similar for a wide variety of applications. This leads to an abundance of boilerplate code in the latter part. For convenience, from now on we will call the part of the program source code responsible for network interactions the *network part* of the program. It is desirable to make the network part of the program as small and simple as possible, because this would allow developers to think less about the network and concentrate on the actual business logic. In a shared-codebase setting this is especially important, because much information is essentially duplicated between the client and server parts of the codebase, and a lot of this duplication can be avoided if the network part is designed with a shared codebase in mind.

However, the standard ways to implement the network part in Kotlin are not well suited for making this part concise in a shared-codebase setting. There are two main families of network technologies used in Kotlin today. The first one is general purpose HTTP frameworks, the most popular of which is Ktor. The second one is RPC frameworks, of which gRPC and Kotlin RPC are notable representatives.

Ktor forces developers to manually write routing logic, serialization and deserialization of request and response bodies, HTTP client calls and error handling code. This gives fine-grained control over the network part logic, which is beneficial when complex public APIs are exposed to the outside world. However, for internal service-to-service communication in shared-codebase projects, this level of control is unnecessary and the manual work is pure overhead.

gRPC and Kotlin RPC reduce this overhead by following the *Remote Method Invocation* (RMI) pattern. The developer declares a service as an interface, implements it in a class and then receives a client proxy for that interface. These frameworks are not boilerplate-heavy in general. In fact, their per function overhead is already small: the developer adds a method signature to the service interface and writes the implementation. The IDL or `@Rpc` interface has an important function in those frameworks. It is the explicit contract between client and server, which is exactly what is needed when client and server are developed independently, in different languages or with different release schedules. In that setting the interface is not boilerplate at all, it is a necessary part of the design.

The picture changes in a shared-codebase setting. Here client and server already share the codebase, data types are shared, and a single change to a signature is a compile-time change for both sides at once. The contract between client and server is implicit in the shared code. Declaring it explicitly in a service interface duplicates information that is

already there. The interface declaration, the implementation class that forwards calls to the actual logic, and the IDL with its proto-to-domain mappings in the case of gRPC, all become redundant. They are boilerplate not because the framework is verbose, but because they exist only to fulfil the framework's structural requirement of having a service interface, while the contract they encode is already encoded in the shared codebase.

Existing RPC frameworks have a few other drawbacks in a shared-codebase setting that follow from the same root cause. Top level functions cannot be remote at all; every remote function has to be a method of some service. At the call site a remote call looks identical to a local one; there is nothing in the syntax that tells the developer that this is a network operation that might be slow or fail. Existing approaches follows a strict OOP style which is not the only viable option for Kotlin. These design choices make sense for gRPC and Kotlin RPC because they are general purpose frameworks aiming at a much wider scope than shared-codebase projects. The same design choices are precisely what makes them carry redundant declarations in a shared-codebase setting.

The goal of the thesis is to develop an alternative approach to organizing network interactions in shared-codebase Kotlin projects that makes the network part smaller and easier by reducing the amount of boilerplate code. We deliberately scope the work to the shared-codebase setting and do not try to compete with gRPC, Kotlin RPC or Ktor outside this scope. The hypothesis is that by sacrificing generality (cross-language support, public-facing API ergonomics, separate evolution of client and server) the framework can give a substantial reduction of boilerplate inside its target scope.

To achieve this goal we look at RPC in Kotlin from another perspective, not common for object-oriented languages. In this thesis we draw inspiration from *coeffects*, a concept from functional programming that captures what a computation requires from its environment to be executed. We do not use coeffects in their formal sense, that is, we do not build on the indexed-comonad calculus of Petricek et al. [19]; we borrow only the informal point of view that "needing a way to reach a remote machine" is a property of a function that belongs in its type. In Kotlin this point of view is naturally expressed using an experimental language feature called *context parameters*, which allows a function to have several receiver parameters that are implicitly passed to the function from the invocation context. We use context parameters to carry information about the target machine. Such a context parameter appears in the remote function signature, which makes remoteness visible on the function declaration, and at the same time is resolved automatically at the call site, so the caller does not have to pass it manually.

1.1 Goal and objectives

The goal of this thesis is to develop an RPC framework specifically for Kotlin Multiplatform shared-codebase projects that reduces boilerplate code compared to existing alternatives in this setting. The framework is not meant to replace general purpose RPC frameworks outside of shared-codebase scope.

To reach this goal we set the following objectives:

1. Prototype an RPC framework that uses context parameters as the mechanism for expressing remote calls.
2. Prototype support for distributed objects, that is, classes with state that lives on a remote machine.

3. Implement the framework as a Kotlin compiler plugin together with a runtime library.
4. Evaluate the result by comparing example projects written using the developed framework with alternatives.

This work has the following structure:

1. Section 2 reviews background and related work on organizing network interactions both in the Kotlin ecosystem and outside it.
2. Section 3 describes the architecture of the developed framework.
3. Section 4 evaluates the framework by using small applications and comparing them with standard approaches.
4. Section 5 discusses limitations of the framework and outlines directions for future work.
5. Section 6 concludes.

2 Background and Related work

This section first introduces RPC in general and the related notion of distributed objects. REST is then covered briefly because it is the other major family of network technologies used in Kotlin. The section then turns to the two RPC frameworks that are the main points of reference throughout the thesis, gRPC and Kotlin RPC. Next it introduces the ideas from functional programming that inspire the design of Kotlin Remote: coeffects, implicit parameters, the Servant RPC framework for Haskell as a concrete example, and multi-tier programming languages. Finally it introduces the Kotlin ecosystem features the framework relies on, including coroutines, Kotlin Multiplatform, `kotlinx.serialization`, Ktor and the experimental context parameters feature.

2.1 RPC

Even though the concept of RPC is well known in the community, it is still better to provide some understanding of RPC that is used in this work.

Remote procedure calls (RPC) — a communication protocol which allows an application executing functions on a remote machine (commonly using a shared computer network) as if these functions were executed on the machine where the application is running.

Remote function — function that can be executed on a remote machine.

The goal of RPC is to hide the details of network communication from an application and make remote function calls look like ordinary function calls. This paradigm allows the programmer to think less about network details and more about the core logic of the application.

The idea of RPC is not new and dates back to the 1970s [30], with enterprise implementations appearing roughly in the 1980s [1].

It is worth noticing that the RPC paradigm has been criticized for trying to hide the complexities of network communication from the developer, which can lead to wrong assumptions about latency and failure modes of remote calls [26]. We are aware of this criticism. As it will be shown later, our framework does not fully eliminate it, since the

remote call expression itself looks the same as a local call. It does, however, push the relevant information into two visible places: the remote function declaration (the context parameter is part of its signature) and the enclosing `context(...)` block at the call site (which has to be opened explicitly to satisfy that requirement). The developer therefore cannot accidentally write a remote call without acknowledging, at the type level, both that it is remote and which remote target it talks to.

2.2 Distributed Objects

RPC does not translate well into the object-oriented world, where interactions between objects residing on different machines are required. That is why, when object-oriented languages became popular, a specific form of RPC was developed. This form is called *RMI*, and it is a way to build communication between *distributed objects* [31] [27].

Distributed Objects — objects that can be accessed from remote machines.

RMI (remote method invocation) — a communication protocol which allows an application executing methods of distributed objects on a remote machine.

The most classic implementations of RMI are CORBA, Java RMI [9] and COM/DCOM. They share many architectural similarities and follow the same general idea of providing distributed objects on a type level [20]. However, the classic forms of RMI in their full form are not popular nowadays. These implementations are not used in modern production code. Modern approaches try to be more lightweight and explicit; they do not try to hide network interactions completely from the developer [23]. The developed framework follows this trend.

2.3 REST

REST (Representational State Transfer) is an architectural style for designing networked applications introduced by Roy Fielding in his PhD thesis [6]. It is built on top of HTTP and uses HTTP methods (GET, POST, PUT, DELETE, etc.) to operate on resources identified by URLs. REST does not specify any particular serialization format, but JSON is the most common one in practice. REST is the most popular way of organizing network interactions in Kotlin today, and is usually implemented using the Ktor framework [16] which is discussed later in section 2.8.4.

In contrast with RPC, REST is centred around resources rather than procedures. From the developer's point of view this means that for each new endpoint the user has to design the URL of the resource, choose the HTTP method that should be used to access it, write routing code on the server, write serialization and deserialization logic for request and response bodies and make the HTTP request from the client. All of this can be tedious, especially when client and server share the same codebase, because in this case much information encoded in the URL and HTTP method is essentially duplicated.

REST gives the developer fine-grained control over network interactions. This is beneficial when network communication is exposed to the outside world (for example, the public API of a service). However, when the network is used for internal communication between microservices that share the same codebase, this fine-grained control is rarely needed and the boilerplate cost dominates.

2.4 gRPC

gRPC [8] is one of the most well-known modern RPC frameworks. It was developed at Google as an open-source successor to an internal RPC system called Stubby [23], which was used internally for more than a decade to connect microservices in Google data centres. Today gRPC supports a wide variety of programming languages, including Kotlin.

Communication in gRPC happens over HTTP/2 and uses Protocol Buffers [22] as the default serialization format. The programmer describes services in a special interface definition language (IDL). For each described service the gRPC tooling generates client and server stubs in the target programming language. The user then implements the server stub on the server side and uses the client stub on the client side to make remote calls. Stubs here have the same function as stubs in Kotlin RPC, which were briefly mentioned in section 3.2.1.

gRPC is a mature framework with little boilerplate per remote function in its target setting. For each remote function the developer writes its description in IDL once and provides an implementation on the server side. The IDL has a clear purpose: it is the cross-language contract between client and server and it is precisely what allows gRPC to support clients and servers written in different languages. In the setting gRPC is designed for, the IDL is not boilerplate; it is the framework's most useful feature.

In a shared-codebase Kotlin setting, however, several gRPC features become redundant. A separate IDL duplicates type information that is already shared between client and server through the codebase. For each remote function the developer has to write its description in IDL, implement the service method that calls the actual function, and provide mappings between IDL types and domain types of the application. The latter is the most visible cost, because in a shared-codebase project the domain types already exist in the shared code and have to be mapped to and from generated IDL types just for the sake of going over the wire. Cross-language support, the feature that justifies all of this in the general case, is not used at all when both sides are Kotlin.

Two more drawbacks are not specific to the shared-codebase setting but matter for our discussion. First, every remote function must be a method of some service; top level functions cannot be remote. Second, a remote call on the client side looks indistinguishable from a local call. This is not entirely good, because remote calls are inherently slower and more error-prone than local ones, so the developer should be aware of when a remote call is happening.

It is worth noticing that gRPC follows the RMI approach: every remote function is a method of some service, and services are first-class entities. This fits naturally with strict OOP languages like Java, but for Kotlin it is not the only option, as we show later in this thesis.

2.5 Kotlin RPC

Kotlin RPC (also known as `kotlinx.rpc`) [14] is a relatively new RPC framework from JetBrains targeted specifically at Kotlin and Kotlin Multiplatform. The general idea of Kotlin RPC is similar to gRPC, but it is more idiomatic for Kotlin. Instead of a separate IDL, the user declares a service as a Kotlin interface annotated with `@Rpc`. The compiler plugin then generates a client stub for this interface and provides means to expose its implementation on the server side. An example of Kotlin RPC usage is shown later in section 3.2.1 in Listing 3.

Compared to gRPC, Kotlin RPC is less verbose for shared-codebase scenarios, because there is no need to write an IDL or mappings between IDL types and the domain types of the application. Kotlin RPC is in fact a thin framework: for each remote function the developer writes a method signature in the `@Rpc` interface and an override in the implementation class. There is almost no other per function overhead. The framework is well-designed and not boilerplate-heavy in any general sense.

What remains, however, is the `@Rpc` interface itself together with its implementation class. As in gRPC, this interface is the contract between client and server. In a separate-codebase project it is a useful thing to have: it is the type that survives both sides being changed independently. In a shared-codebase Kotlin project the contract is implicit in the shared code and the `@Rpc` interface duplicates it. The implementation class on the server only forwards calls to the actual logic, which in a shared-codebase project lives in the same module anyway, so this class is essentially a layer of indirection that exists only because Kotlin RPC requires it. The same fundamental drawbacks as in gRPC also remain: top level functions cannot be remote, every remote function has to be a method of some service, and remote calls are not visible at the call site. Just like gRPC, Kotlin RPC is essentially an RMI framework, which is not the only viable approach for Kotlin.

2.6 Coeffects

The approach used in the Kotlin Remote framework is heavily inspired by ideas from functional programming, specifically by the concept of *coeffects* [19][3].

Coeffect — property of a computation that describes what this computation requires from its environment in order to be executed.

Coeffects can be thought of as dual to *effects*. While effects describe what a computation does to its environment (for example, mutating a global variable, throwing an exception or performing IO), coeffects describe what a computation requires from its environment to be executed (for example, access to a specific resource, the presence of a certain configuration value or, in our case, a way to reach a remote machine). Petricek et al. [19] introduced a unified static analysis for context-dependence that captures several seemingly different examples, including liveness analysis, implicit parameters and dataflow caching, under one umbrella. The theoretical foundation of coeffects relies on indexed comonads, but in practice coeffects can be expressed using more familiar language constructs.

It is useful to look closer at the duality between effects and coeffects. Effects in their modern form are usually expressed via *algebraic effects* [21] and *effect handlers*. Algebraic effects describe operations that a computation can perform on its environment (like throwing an exception, reading from a file or sending a network request) and effect handlers describe how these operations should be interpreted. Languages like Koka [17] make algebraic effects central to their type system. The duality is the following: effects describe what a computation does to the environment and how this is interpreted, while coeffects describe what a computation requires from the environment and how this requirement is satisfied. A remote call can be viewed both as an effect (it performs network I/O) and as a coeffect (it requires a way to reach the remote machine). Kotlin Remote takes the coeffect view, but only informally: we do not introduce an effect system or any of the comonadic machinery of [19]. We borrow only the design intuition that the requirement "I need a way to reach the remote machine" should be a part of the function type, and rely on the existing Kotlin context-parameter feature to make this requirement explicit.

The most well-known practical encodings of coeffects in functional languages are *implicit parameters* [18] and the *Reader monad*. Implicit parameters are parameters of a function that are not passed explicitly at the call site, but are resolved automatically from the surrounding context. The Reader monad threads a read-only environment through a computation in a way that does not require explicitly passing it to every function. From the user's point of view both mechanisms allow expressing context-dependence of a computation in a type-checked way.

The Reader monad is used by an RPC framework called Servant [24] for the Haskell programming language to implement remote calls. Listing 1 shows a small Servant client.

```
position :: Int -> Int -> ClientM Position
hello   :: Maybe String -> ClientM HelloMessage

queries :: ClientM (Position, HelloMessage)
queries = do
  pos <- position 10 10
  message <- hello (Just "servant")
  return (pos, message)

run :: IO ()
run = do
  Right (pos, message) <- runClientM queries ... "localhost"
  print pos
  print message
```

Listing 1: Remote function call in Servant

Here `position` and `hello` are remote functions and their return type is `ClientM x`, where `x` is the value the function computes and `ClientM` is a monad. Inside the `queries` function several remote calls are made. The place where information about the target machine is provided is `runClientM queries ... "localhost"`. `ClientM` is essentially a Reader monad: it provides the configuration with which remote functions are run.

This functional approach has several properties that contrast with the stub-based OOP approach described later. Top level functions can be remote, no interface declaration is needed, every remote function is clearly marked as remote at the type level via its `ClientM` return type, and there is no need to construct several stubs to call the same function on several target machines: a different configuration is passed to `runClientM` instead. The trade-off is that the code is locked into the `ClientM` monad and has to be lifted in and out of it explicitly, a price that Kotlin can largely avoid because context parameters express the same dependence using ordinary function syntax. Kotlin Remote uses similar design: where Servant uses the Reader monad, Kotlin Remote uses a context parameter; where Servant has `runClientM`, Kotlin Remote has the `context(config) { ... }` block.

The same principle, when applied to Kotlin, gives the framework that is presented in this thesis. In Kotlin Remote, the analogue of the Reader monad and of Haskell-style implicit parameters is Kotlin *context parameters*, which are described later in section 2.8.5. The context parameter of a remote function is a coeffect in the informal sense described above: it states that, in order to execute this function, the caller has to provide a way to reach the remote machine. The requirement is visible in the function signature and, just like in Servant, can be satisfied either by a configured remote context (and then a remote call

is performed) or by a local context (and then the function body is executed locally on the server).

2.7 Multi-tier programming languages

The idea of distinguishing on a type level where a particular piece of code is executed (client or server) is not new and was explored by several programming languages designed specifically for web applications. Such languages are often called *multi-tier* or *tierless*, because they allow writing client and server code in the same language and the same codebase, while still keeping a clear separation between the two on a type level. Multi-tier languages are tightly related to the coeffects discussed in the previous section: the location of code execution is essentially a coeffect, because the computation requires from the environment a particular machine on which to be executed.

Examples of such languages include Links [4], Ur/Web [2] and Opa [28]. In Links, for example, a function annotation can specify whether the function should be executed on a client or on a server. The compiler then checks that all the calls between client and server functions are done correctly and generates code that handles network communication automatically. Ur/Web takes a similar approach but additionally provides type-safe SQL queries and HTML templates. Opa goes even further by combining client, server and database code in a single language with a unified type system.

The approach taken in Kotlin Remote can be seen as a library-level approximation of what multi-tier languages do at the language level. Instead of introducing a new language with a dedicated type system for tiering, we use an existing language feature (context parameters) to encode the location of code execution on a type level. This approach is less powerful than a full multi-tier language, because it is limited by what can be expressed using context parameters. But it has its own advantages: it can be used in existing Kotlin projects without rewriting them in a new language, and it integrates naturally with the rest of the Kotlin ecosystem.

2.8 Kotlin Ecosystem

The framework developed as part of this thesis uses many technologies from the Kotlin ecosystem. Therefore, it makes sense to briefly introduce these technologies.

2.8.1 Kotlin Coroutines

Coroutines [5] is a Kotlin feature for asynchronous programming. Functions that can be paused and resumed without blocking the executing thread are called *suspend functions* in Kotlin and have a special `suspend` modifier. Suspend functions can only be called from other suspend functions or from special coroutine builders like `runBlocking` or `launch`. This restriction is sometimes called the *function coloring* problem [29][10], because suspend functions and ordinary functions are two distinct kinds (or *colors*) of functions in Kotlin. For our purposes function coloring is beneficial, because remote calls are inherently asynchronous, so it is natural for remote functions in Kotlin Remote to be suspend. Internally, coroutines are implemented using continuations. Suspend functions are transformed by the Kotlin compiler into state machines that take a continuation as an additional parameter.

2.8.2 Kotlin Multiplatform

Kotlin Multiplatform (KMP) [13] is a technology that allows writing code that can be compiled to several platforms, including JVM, JavaScript, Native (iOS, Linux, macOS, Windows) and WebAssembly. Kotlin code that targets KMP is split into several so-called *source sets*. There is a common source set with code shared between all the platforms and several platform-specific source sets with code that is specific to a particular platform.

KMP introduces the `expect/actual` mechanism for declaring code that has a different implementation for different platforms. In the common source set the developer declares a function or class with the `expect` keyword, and this declaration has no body. In each platform-specific source set the developer provides the actual implementation of this declaration using the `actual` keyword. When the common source set is compiled for a particular platform, the `expect` declaration is replaced with the corresponding `actual` one.

One important limitation of KMP is that it does not support fully fledged reflection in the common source set. The JVM target supports a rich reflection API, but other targets either do not support reflection at all or support only a very limited subset of it. This means that libraries that target KMP cannot rely on reflection to gather information about types and functions at runtime. Instead, such information has to be gathered at compile time, usually using compiler plugins. Kotlin Remote targets KMP and faces this limitation, which shaped many architectural decisions described later in section 3.3.

2.8.3 `kotlinx.serialization`

`kotlinx.serialization` [15] is the standard library for serialization in Kotlin. It supports several formats, including JSON, CBOR and Protocol Buffers. Serialization logic for user-defined classes is generated by a Kotlin compiler plugin from class declarations marked with the `@Serializable` annotation. This means that serialization works without reflection and, as a consequence, works in all Kotlin Multiplatform targets. Kotlin Remote uses `kotlinx.serialization` to serialize and deserialize remote function arguments and return values.

2.8.4 Ktor

Ktor [16] is a framework for building asynchronous servers and clients in Kotlin. It is built on top of Kotlin Coroutines and is a part of the Kotlin ecosystem maintained by JetBrains. Ktor follows a plugin-based architecture: most of the functionality (authentication, content negotiation, logging, etc.) is provided by plugins that the user installs explicitly. Kotlin Remote uses Ktor as the default transport: the `core-ktor` module provides a Ktor plugin called `KRemote` that handles remote calls on the server side and an integration of `HttpClient` for the client side. Although Ktor is used as the default transport, the core of Kotlin Remote is transport-agnostic; alternative transports can be plugged in by providing a custom implementation of the `RemoteClient` interface.

2.8.5 Kotlin context parameters

Context parameters [11] is an experimental Kotlin language feature that introduces implicit parameters to Kotlin. A function with context parameters declares its requirements from the surrounding context using a special `context(...)` block before the function declaration. Listing 2 shows a simple example of a function with a context parameter.


```

context(logger: Logger)
fun greet(name: String) {
    logger.info("Hello, $name")
}

fun main() {
    val logger = Logger()
    context(logger) {
        greet("World")
    }
}

```

Listing 2: Function with context parameter

In this example the function `greet` has a context parameter `logger` of type `Logger`. When `greet` is called, `logger` is not passed explicitly, instead it is resolved automatically from the enclosing `context(...)` block. If there is no such block in the lexical scope of the call, a compilation error is raised. Context parameters can be of any type and there can be several of them. They are similar to implicit parameters from Scala or Haskell [18], and just like them allow expressing coeffects in Kotlin in a typed way.

In Kotlin Remote context parameters are used to provide remote functions with information about the target machine. This way information about remote configuration is propagated through the call chain implicitly, while at the same time being visible in the type signature of the remote function. A detailed description of how exactly context parameters are used in Kotlin Remote is provided later in section 3.2.1.

3 Kotlin Remote Implementation

This section describes the implementation of the framework developed in this thesis, called *Kotlin Remote*. The presentation has five parts. First, section 3.1 gives a high-level overview of the framework's module structure, which is referred to throughout the rest of the section. Second, section 3.2 describes how a single remote function is expressed on the client and on the server. The same subsection also explains how the same declaration can be used for both roles in a shared codebase via the `RemoteContext` type and how subtyping between configurations gives rise to hierarchical tiers of capabilities. Third, section 3.3 explains the compiler-plugin code generation that reduces the per-function user code, including the `@Remote` body transformation and the `genCallableMap` intrinsic. Fourth, section 3.4 describes how exceptions thrown on the server are propagated to the client. Finally, sections 3.5 and 3.6 cover the default Ktor transport and the support for stateful remote classes with leased distributed garbage collection.

3.1 Module structure

Figure 1 shows the Gradle module dependency graph of Kotlin Remote. Each rectangle is a Gradle module. The `compiler-plugin` module contains the Kotlin compiler plugin that performs all the code generation described later in section 3.3. The `core-functions` module contains the runtime library for remote functions: the `RemoteContext` and `RemoteConfig` types together with the `RemoteCall` and `RemoteCallable` data structures, plus the abstract

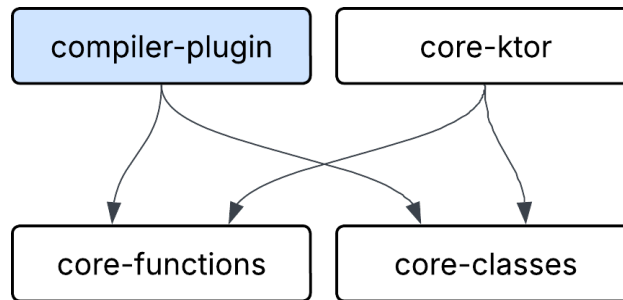


Figure 1: Module dependency graph

`RemoteClient` interface that decouples the framework from a concrete transport. The `core-classes` module sits on top of `core-functions` and adds support for stateful remote classes (distributed objects), including the stub mechanism and the leased distributed garbage collector described in section 3.6. The `core-ktor` module provides the default transport implementation. It ships an `HttpClient`-based `RemoteClient` on the client side and a Ktor server-side `KRemote` plugin together with the lease HTTP routes on the server side. `core-classes` and `core-ktor` are independent of each other; an alternative transport can be plugged in by providing a custom `RemoteClient` without touching either module.

3.2 Remote functions

A function that can be executed on a remote machine is the central abstraction of any RPC framework, and Kotlin Remote is no exception. We will call such a function a *remote function*. To reason about remote functions, two terms have to be introduced: client and server. In the RPC world the *client* is the machine from which the remote call is made and the *server* is the target machine of this remote call, the machine where the call is actually executed.

3.2.1 Client

Let us take a look at how a remote call is done on the client in Kotlin Remote. In order to call a function on a remote machine it is necessary to pass to this function some information specific to the remote call, including the address of the target machine and a way to serialize function arguments and deserialize its result. In existing RPC frameworks this information is provided to the function using a stub. To better understand what a stub is and how it provides the necessary information for a remote call, let us take a look at the example for Kotlin RPC in Listing 3. The user has to create an instance of `KtorRpcClient`, which is configured with the remote host URL and the serialization schema. Then using this client the user can create the service stub `client.withService<PizzaShop>()`, where `PizzaShop` is the interface whose implementation is generated automatically by Kotlin RPC. Such an interface is usually called the *service* that the remote machine provides.

The stub `pizzaShop` uses the client with which the stub was created to make remote calls. It has some drawbacks:

1. Top level functions cannot be called using this approach.
2. The user has to declare an interface just for remote functions.


```

@Rpc
interface PizzaShop { suspend fun orderPizza(pizza: Pizza): Receipt }
fun main() = runBlocking {
    val client: KtorRpcClient = HttpClient { ... }.rpc {
        url { host = "localhost" }
        serialization { json() }
    }

    val pizzaShop: PizzaShop = client.withService<PizzaShop>()

    pizzaShop.orderPizza(Pizza("Pepperoni"))
}

```

Listing 3: Remote function call in Kotlin RPC

3. When calling methods of the stub the user does not know that the call results in a network interaction; it is completely implicit.
4. When the user wants to call remote functions of the same service on several target machines, several stubs have to be created.

gRPC uses virtually the same approach as Kotlin RPC but is a bit more cumbersome.

Stubs are not the only way to provide a remote function with the information it needs. The alternative used in functional programming was described in section 2.6 on the example of *Servant* [24]: a function expresses its dependence on the remote configuration in its type (through the `ClientM` return type), and that configuration is provided at the top of the call chain (through `runClientM`) instead of being baked into a stub. Kotlin Remote takes the same approach, with the role of the Reader monad played by Kotlin's experimental *context parameters* [11] feature: a remote function declares a context parameter for the remote configuration, the configuration is provided implicitly by an enclosing `context(config) { ... }` block, and at every call site the dependence is visible at the type level even though the call expression itself looks like an ordinary function call.

Let us imagine we want to execute some function `multiply` remotely. Listing 4 shows the client implementation for it.

```

fun pass(): Nothing = error("pass")
fun multiply(lhs: Long, rhs: Long): Long = pass()

```

Listing 4: Client part of multiply function

The body of the function is undefined, because it is not needed on the client. To pass to the `multiply` function a way to communicate with the remote machine, we can add a context parameter to `multiply`, as shown in Listing 5. The reader may notice that it is inconvenient to write `c.client.call(...)` for every remote function. This problem will be addressed later in section 3.3 when the code generation done by the framework will be discussed. In Listing 5 the remote client contains all the information needed for a remote call, including the target address and the serialization schema. The default implementation of `RemoteClient` uses the *Ktor* [16] `HttpClient`, which is set up by the user. In order to call `multiply` the user can write the code shown in Listing 6.

```

interface RemoteConfig {
    val client: RemoteClient
}

context(c: RemoteConfig)
suspend fun multiply(lhs: Long, rhs: Long): Long {
    return c.client.call<Long>(
        RemoteCall("multiply", arrayOf(lhs, rhs))
    )
}

```

Listing 5: Multiply function with context

```

data object CalculatorRemoteConfig : RemoteConfig {
    override val client: RemoteClient = HttpClient {
        defaultRequest { url("http://localhost:8080") ... }
        install(ContentNegotiation) {
            json(Json {
                serializersModule = remoteSerializersModule {
                    callableMap = CallableMap(...)
                }
            })
        }
    }.remoteClient("/call")
}

fun main() = runBlocking {
    context(CalculatorRemoteConfig) {
        println(multiply(6, 1))
    }
}

```

Listing 6: Remote function call

The setup of `HttpClient` is standard, the interesting part is the `remoteSerializersModule` function. This function declares the serialization schema for remote function parameters and result. It returns an instance of `SerializationModule` from `kotlinx.serialization` [15]. This `SerializationModule` has serializers to serialize and deserialize remote function parameters and return values. `RemoteClient.call` accepts a `RemoteCall` from Listing 7. The field `callableName` is the name of the function that is called and `arguments` are the arguments with which this function is called. This class has a custom serializer that serializes `arguments: Array<Any?>` using `callableName`. This serializer is created by `remoteSerializersModule`. To implement this custom serializer, information about the remote function with name `callableName` is needed. The only way to get this information in Kotlin Multiplatform (KMP) is by asking the user to provide it or by gathering it automatically. The problem is that this information cannot be gathered at runtime because KMP (unlike Java or Kotlin JVM) does not support fully fledged reflection. This means that we cannot get a function signature at runtime or a type by its name. But this is exactly the information needed for serialization. For now, we assume that the required information is gathered somehow and stored inside a structure called `CallableMap`. This is a map from the fully

qualified name of a remote function to information about this function, represented by `RemoteCallable` from Listing 7.

The pipeline is the following: `remoteSerializersModule` creates a serializer for `RemoteCall` using `CallableMap`. This serializer uses `CallableMap` and `RemoteCall.callableName` to determine the types of parameters in `RemoteCall.arguments` and serialize them. On the server `RemoteCall` is deserialized in a similar manner. The server implementation will be discussed in detail in the next section.

```
class CallableMap(val callableMap: Map<String, RemoteCallable> = mapOf())

class RemoteCall(
    val callableName: String,
    val arguments: Array<Any?>,
)

data class RemoteCallable(
    val returnType: KType,
    val parameterTypes: Array<KType>,
)
```

Listing 7: `RemoteCall` and `RemoteCallable` class

The simplest way to get a `CallableMap` is to ask the user to write it manually. Let us assume for now that the user has to provide information about all the remote functions by describing `CallableMap`. This problem will be addressed later in section 3.3.2.

3.2.2 Server

It was described how to use remote functions from the client side. Now it is time to understand how to handle remote calls on the server side. When a remote function is called from the client, an HTTP request is sent to the server. For example, for the code from Listing 6 the HTTP request will be sent to the URL "`http://localhost:8080/call`", there will be no query parameters and the payload will be a serialized version of `RemoteCall`. For this specific example it will be in JSON format. Here it is worth noticing that even though the framework was written with HTTP in mind, any transport protocol could be used. To do that, an alternative implementation of the `RemoteClient` interface should be provided.

The user could write a handler for such a request, either using Ktor or anything else. But to make this process easier, the framework provides default handlers for remote calls. To use them, a Ktor server with the installed `KRemote` plugin should be created. This can be done by writing the code shown in Listing 8. This will run an HTTP server on the URL "`http://localhost:8080/call`". Just like on the client, `CallableMap` is used to describe the remote functions this server can handle.

The problem is that the server, as opposed to the client, should not only serialize and deserialize function arguments and return value, but also invoke the function itself. To do that, `RemoteCallable` should be extended with a field of type `RemoteInvokator` (Listing 9). `RemoteInvokator` is a reference to the remote method. It is easier to understand by example in Listing 10, which shows `RemoteCallable` for the `multiply` function. Of course,

```

fun main() {
    embeddedServer(Netty, port = 8080) {
        install(CallLogging)
        install(KRemote) {
            callableMap = CallableMap(..)
        }
        routing {
            remote("/call")
        }
    }.start(wait = true)
}

```

Listing 8: Kotlin Remote server

this multiply function should have a normal body and no context parameters, as opposed to the client version.

```

data class RemoteCallable(
    val returnType: RemoteType,
    val invokator: RemoteInvokator,
    val parameters: Array<RemoteParameter>,
)

fun interface RemoteInvokator {
    suspend fun call(parameters: Array<Any?>): Any?
}

```

Listing 9: RemoteCallable with invokator

```

RemoteCallable(
    returnType = typeOf<Long>(),
    invokator = { multiply(it[0] as Long, it[1] as Long) },
    parameters = arrayOf(typeOf<Long>(), typeOf<Long>()),
)

```

Listing 10: RemoteCallable for multiply

Now the server can invoke RemoteInvokator when it receives a RemoteCall. To do that the server finds the RemoteCallable in CallableMap by callableName from RemoteCall.

3.2.3 Unified client and server bodies via RemoteContext

The methods described above for creating client and server are not very good when client and server share the same codebase. By that we mean that client and server code lives in a single Gradle module. Gradle [7] is the most popular build system used by Kotlin; Kotlin does not have its own concept of modules. The idea is that using the same function for client calls and server handling is more convenient than using two functions with the same signature. However, the bodies of the remote functions differ between the client point of view and the server point of view. In order to fix that we can use more elaborate context parameters. A "switch" between server and client body of the remote function is needed, and such a switch can be encoded using the same context

parameter that was used for the remote call. Listing 11 shows a modified version of the multiply function. RemoteContext is a sum of types: it is either LocalContext or ConfiguredContext. Notice how the covariant type parameter T and the type Nothing are used here. Because Nothing is the bottom type in the Kotlin type system (it is a subtype of every type) and T is declaration-site covariant (`out T`), LocalContext (which extends RemoteContext<Nothing>) is a subtype of RemoteContext<T> for every T. It follows that any remote function declared with context(_: RemoteContext<X>) for any X can be called in a LocalContext, and the call will type-check. When a remote function is called in a LocalContext it is executed in the current coroutine and does not result in a request to the server. Because of that the same function can be called on the server in a local context and on the client in a configured context.

```
sealed interface RemoteContext <out T: RemoteConfig>
object LocalContext: RemoteContext<Nothing>
class ConfiguredContext<T: RemoteConfig>(val config: T): RemoteContext<T>

context(ctx: RemoteContext<RemoteConfig>)
suspend fun multiply(lhs: Long, rhs: Long) =
    if (ctx is LocalContext) {
        lhs * rhs
    } else {
        (ctx as ConfiguredContext<RemoteConfig>).config.client.call<Long>(
            RemoteCall("multiply", arrayOf(lhs, rhs))
        )
    }
}
```

Listing 11: Updated multiply

Now RemoteInvokator should be changed to reflect this duality of remote functions. The added context parameter signals that a remote function can only be called locally on a server. The server now has to provide LocalContext for all the invoker calls.

```
fun interface RemoteInvokator {
    context(_: LocalContext)
    suspend fun call(parameters: Array<Any?>): Any?
}
```

Listing 12: Updated RemoteInvokator

3.2.4 Remote context relations

Contexts of remote functions can be used to express complex relations between capabilities of different servers using subtyping. For example, let us imagine that there are two servers for arithmetic operations. One server only provides basic operations like multiplication, while the other also supports trigonometric functions. That can be expressed on a type level in the way shown in Listing 13. Clients here form a hierarchy. Notice how mul and divide can be called from the sin function. Furthermore, calls to mul and divide from sin do not result in a remote call, because everything is done in the LocalContext provided by the invoker on the server. However, if the user really wants to call functions marked with ArithmeticConfig from inside sin remotely, the context can be changed by calling context(ArithmeticConfigImpl) { ... }.

```

interface ArithmeticConfig: RemoteConfig
object ArithmeticConfigImpl: ArithmeticConfig { ... } // localhost:8001
object TrigonometricConfig: ArithmeticConfig { ... } // localhost:8002
context(_: RemoteContext<ArithmeticConfig>)
suspend infix fun Double.mul(rhs: Double) = this * rhs
context(_: RemoteContext<ArithmeticConfig>)
suspend infix fun Double.divide(rhs: Double) = this / rhs
context(_: RemoteContext<TrigonometricConfig>)
suspend fun sin(x: Double): Double {
    var (term, sum, n) = Triple(x, x, 1.0)
    while (abs(term) > 1e-5) {
        val divisor = (2.0 mul n) mul (2.0 mul n + 1.0)
        term = term mul -1.0 mul x mul x divide divisor
        sum = sum + term
        n++
    }
    return sum
}

```

Listing 13: Hierarchy of clients

3.3 Remote functions code generation

Some of the actions required from the user in order to use remote functions can be delegated to the compiler using code generation. In particular, `CallableMap` and calls to contexts of remote functions can be generated by the compiler. There are several standard ways to generate code in Kotlin. Kotlin Remote uses the most powerful one: compiler plugins.

The Kotlin compiler architecture consists of two main parts: backend and frontend. Kotlin compiler plugins can also be separated into two categories: frontend plugins and backend plugins. Frontend plugins allow extending the behavior of the Kotlin compiler frontend. Backend plugins do the same thing but for the compiler's backend. Both the compiler's frontend and backend consist of a series of so-called lowerings, transformations of the internal representation (IR) of the Kotlin source code inside the compiler. Plugins allow adding new lowerings; some of the lowerings do not change the IR and only check for certain invariants.

The main difference between frontend and backend plugins is that frontend plugins only allow adding new definitions and changing attributes of the old ones, but they do not allow changing the bodies of definitions. However, the results of plugins' work on the frontend can be seen inside the IDE, which makes the frontend a good place to add verification, for example checking that all the remote functions are suspend. Some code generation can also be done on the frontend more easily than on the backend. Because of that, Kotlin Remote generates code on both the frontend and the backend. It is also worth noticing that the backend of the Kotlin compiler has a less stable API. So it is generally a good idea to make changes on the frontend whenever possible.

Nevertheless, the Kotlin Remote plugin generates almost all the code in backend plugins, because they are much more powerful. This is why from now on we will focus mainly on the backend part of the Kotlin Remote compiler plugin.

3.3.1 Remote function body generation

Firstly, there is a small problem with using a compiler plugin. It is hard to distinguish remote functions from normal ones. A context parameter could be used to do that, but it is a bad approach because not every function with a context parameter of type `RemoteContext` should be remote. Sometimes we want to add a context parameter to a function to mark that this function can call remote functions, but this should not imply that the function itself is remote. Let us take a look at the example in Listing 14. The implementation of `multiply` is taken from Listing 11. Here the `power` function has a context parameter of type `RemoteContext` but is not itself remote, its body is executed locally; the context parameter only shows that `power` is capable of calling remote functions like `multiply`. The context parameter on `power` signals that it has a *coeffect*, in other words it requires from the environment a way to make remote calls.

To express such coeffects we need a separate way to mark remote functions. In Kotlin Remote the `Remote` annotation is used for that. In Listing 14 one can notice that the `multiply` function has such an annotation. Every function marked with `Remote` undergoes a body transformation that is exactly the same as the transformation of `multiply` that was done manually in Listing 11. The name of the remote call is the fully qualified name of the function. It is worth noticing that a function marked with the `@Remote` annotation has to have a context parameter of type `RemoteContext`, otherwise an error is raised by the Kotlin Remote frontend compiler plugin. This error is visible in IntelliJ IDEA.

```
@Remote
context(_: RemoteContext<RemoteConfig>)
suspend fun multiply(lhs: Long, rhs: Long) = ...

context(_: RemoteContext<RemoteConfig>)
private suspend fun power(base: Long, power: Int): Long {
    return generateSequence { base }
        .take(power)
        .fold(1L) { acc, x -> multiply(acc, x) }
}
```

Listing 14: Local function with `RemoteContext`

3.3.2 Callable map generation

At the end of section 3.2.1 we mentioned that there is a problem with the user having to describe the `CallableMap` manually. Kotlin Remote addresses this issue by automatically generating the callable map using a function called `genCallableMap` from Listing 15. As one can see, this function has no body. This is because every call to this function is substituted with a `CallableMap` instance generated by the compiler plugin. One can say that `genCallableMap` is a macro (like in C), but a very smart one. The Kotlin Remote compiler plugin scans the source code during compilation, finds each function marked with `@Remote` and for each such function generates an entry in the `CallableMap` describing this function. All the information needed is accessible at compile time. This means that the callable map generated by `genCallableMap` can have many entries, because functions from all over the source code are gathered.

It is worth noticing that `genCallableMap` adds every `@Remote` function to the callable map

regardless of its Kotlin visibility, including functions with non-public visibility. This means that by using `genCallableMap` it is possible to invoke such functions over the network, so the `@Remote` annotation, rather than the Kotlin visibility modifier, is the real boundary of the server's network API. We recommend keeping every `@Remote` function `public` so that Kotlin visibility matches the actual network exposure. A frontend compiler plugin check that forbids lower-than-`public` visibility on `@Remote` declarations would make this convention enforceable mechanically. We did not implement such a check because the planned remote lambdas extension (section 5.2) will need to dispatch closure-captured callables that are not public top-level Kotlin functions, so a strict visibility rule would have to be revisited together with that work.

Another quirk of `genCallableMap` is that it does not generate entries for remote functions from dependencies, because the source code of dependencies is not accessible (it is already compiled). To alleviate that, dependencies can save their callable maps generated by `genCallableMap`, for example in a public static property. This way the dependent code can use these saved callable maps, for example by merging them with its own callable map.

```
fun genCallableMap(): CallableMap =  
    throw IllegalStateException("Intrinsic function was called directly")
```

Listing 15: `genCallableMap` function

3.4 Exception handling

During execution of remote functions on remote machines exceptions can occur. Such erroneous situations should be handled by the client that made the remote call. Here a difficult decision has to be made. We have two options: hide the fact that the exception occurred on the remote machine, or make it obvious. Both options have their benefits and drawbacks, but we decided to opt for the second approach, because we wanted to make calls to remote functions more seamless. So in the current implementation an exception thrown on the remote machine can be caught on the client as if the function were called locally. Listing 16 shows an example. This code outputs the exception from Listing 17. One can see that there are no frames in the exception related to the remote call, except the synthetic stack frame `== Remote Call Boundary ==` that indicates the boundary between stack frames from the remote machine and local frames.

```
@Remote  
context(_: RemoteContext<RemoteConfig>)  
suspend fun div(x: Int, y: Int) = x / y  
  
fun main() {  
    try {  
        ServerConfig.runWith { div(10, 0) }  
    } catch (e: ArithmeticException) {  
        e.printStackTrace()  
    }  
}
```

Listing 16: Remote exception handling


```

java.lang.ArithmeticException: / by zero
    at proposalExamples.ExceptionKt.div(Exception.kt:11)
    at == Remote Call Boundary ==.(Unknown Source)
    at proposalExamples.ExceptionKt.main(Exception.kt:15)
    ...

```

Listing 17: Remote exception content

To implement this, a remote function must first be able to return either a result or an error. In Kotlin this can be expressed as shown in Listing 18.

```

sealed interface RemoteResponse<T> {
    data class Success<T>(val value: T): RemoteResponse<T>
    data class Failure(val error: Throwable): RemoteResponse<Nothing>
}

```

Listing 18: Return type of remote functions

However, two more things must be implemented: serialization of exceptions and altering stack traces of exceptions. Exceptions are not serialized by default using `kotlinx.serialization`, the library that Kotlin Remote uses for serialization. To make exceptions serializable they should be registered as polymorphic subclasses of `Throwable`. This is already done for all exceptions from the standard Kotlin Multiplatform library. When the user wants to throw a custom exception, they should register it in a `SerializersModule` using the `throwableSerializer` function, as shown in Listing 19. When an exception is not registered it is serialized as `UnregisteredRemoteException`. To alter stack traces in KMP, `expect/actual` functions are used. Only the JVM backend allows changing exceptions. For all other backends the exception from the remote machine is wrapped in a `RemoteException` as a cause.

```

SerializersModule {
    polymorphic(Throwable::class) {
        subclass(MyException::class, throwableSerializer(::MyException))
    }
}

```

Listing 19: Custom exception registration

3.5 Ktor integration

Kotlin Remote is meant to be transport agnostic. This is done by splitting code into three modules: `core-functions`, `core-classes` and `core-ktor`. Until now we have almost exclusively been discussing the `core-functions` module that contains logic related to remote functions. Now let us briefly discuss the `core-ktor` module, which contains the Kotlin Remote integration with Ktor. Ktor is a mature framework that offers many features like authentication support and logging. Kotlin Remote used together with Ktor supports the same functionality Ktor does. For example, authentication can be supported using the code in Listing 20. Notice the call to the `handleRemoteCall` function inside the server. This function passes control to the `core-ktor` module, which deserializes the arguments of the remote call, invokes the remote function and serializes and sends the result to the client. Inside the `/call` route `handleRemoteCall` is called implicitly.

```

val server = embeddedServer(Netty, port = 8080) {
    install(Authentication) {
        basic("auth-basic") { validate { ... } }
    }
    install(KRemote) { callableMap = genCallableMap() }
    routing {
        authenticate("auth-basic") {
            post("/callAuth") {
                println("Welcome to the protected route!")
                handleRemoteCall()
            }
        }
        remote("/call")
    }
}

data object AuthServerConfig : RemoteConfig {
    override val client: RemoteClient = HttpClient {
        ...
        install(Auth) { ... }
    }.remoteClient(genCallableMap(), "/callAuth")
}

```

Listing 20: Authentication support

3.6 Remote classes

Because Kotlin is not only a procedural and functional language but also relies heavily on OOP, it was decided to add support for calling methods of classes remotely, not only top-level functions. Basic support for that is achieved easily. The only important difference between top level functions and methods is that methods have an implicit `this` parameter. In our case we can think about the `this` parameter just like any other regular parameter. So calling a remote method serializes `this` and the other method parameters on the client, sends the call to the server, the server deserializes `this` and the other parameters and calls the method on the deserialized `this`. The good news is that this is already supported without any additional effort, because the Kotlin compiler treats `this` as a regular parameter of the function. But there are two large problems with this approach. Firstly, `this` has to be serializable, and the serialization logic has to be provided by the user. Secondly, methods that mutate `this` will mutate the deserialized copy of `this`, which means they will have no effect on the original `this`.

These problems are not new and were addressed in many existing RMI frameworks, including Java RMI. In Kotlin Remote we decided to use a well-known approach to solve them: stubs. Classes in Kotlin Remote can be marked with the `@RemoteSerializable` annotation. For each remote serializable class a custom `kotlinx.serialization` serializer is generated.

This custom serializer serializes a remote class as a stub. A stub is a subclass of a remote class that is generated for every remote class. The stub has two fields: the ID and the URL of the machine it was created on. During serialization of a remote class it is added to a special map called `RemoteInstancesPool`. Keys in this pool are stub IDs and values are

the original classes.

A stub is deserialized either as the original class, if it is deserialized on the machine it was serialized on and `RemoteInstancesPool` contains it, or as a stub. To determine whether the stub was serialized on a certain machine, the URL of this machine is compared with the URL from the stub.

This URL is only used for disambiguation of the nodes and for garbage collection (more about it later in section 3.7). This URL is not used to make remote calls at all; the address of the target machine is still received from the context parameter of remote functions. The URL is specified on the server when the application is configured. This means that the user should know the address of the machine the server is running on.

Stubs are "remote references" that are supported by Kotlin Remote without altering the logic related to remote functions. Only the serialization functionality is changed. Because of that, logic related to remote classes lives in a separate Gradle module named `core-classes`, which neither depends on nor is depended on by the `core-functions` module described earlier. It is worth noticing that stubs are serialized as is, which allows passing remote references from one machine to another even if the reference was created on a third machine.

3.6.1 Remote classes code generation

For each `@RemoteSerializable` class a stub subclass is generated. This subclass is generated inside the original class as shown in Listing 21. `RemoteClassStub` extends `Calculator` and implements the `Stub` interface. The `Stub` interface can be used by the user to check whether a certain instance of `Calculator` is a stub or not. Notice how `RemoteClassStub` invokes `Calculator`'s default constructor even though `Calculator` does not have one. This is achieved by generating a default constructor for `Calculator` in the Kotlin Remote backend compiler plugin. `Calculator` is also made open implicitly by the same plugin on the backend. Because these changes are made on the backend, the `Calculator` class cannot be extended, and its default constructor cannot be called from user defined Kotlin code. Only code generated by the compiler plugin can do that.

```
@RemoteSerializable
class Calculator(private var init: Int) {
    @Remote
    context(_: RemoteContext<RemoteConfig>)
    suspend fun multiply(x: Int): Int {
        init *= x
        return init
    }
    class RemoteClassStub(
        override val id: Long,
        override val url: String
    ) : Calculator(), Stub
}
```

Listing 21: Remote class stub

Without reflection (which we cannot use because of KMP) it is unclear how to deserialize `Calculator` as its `RemoteClassStub`, because every remote class has its own

RemoteClassStub type. To solve this issue another Kotlin Remote intrinsic is used, called `genRemoteClassList`. Listing 22 shows it. The list of `RemoteClassDescriptor` is used to generate serializers and deserializers for remote classes using contextual serialization from `kotlinx.serialization`.

```
data class RemoteClassDescriptor<T : Any>(
    val clazz: KClass<T>,
    val serialName: String,
    val stubFabric: (Long, String) -> T,
)

fun genRemoteClassList(): List<RemoteClassDescriptor<Any>> =
    RemoteIntrinsic
```

Listing 22: `genRemoteClassList` intrinsic

3.7 Garbage collection

So far we have described no way to remove remote classes from `RemoteInstancesPool`. This is a serious resource leak: every time a remote class is serialized on the server a new entry is added to the pool and it stays there forever. To resolve this issue several techniques can be used, including reference counting or full-blown distributed garbage collection. We decided to opt for a leasing approach because it is simple and durable. The general idea is that on the server every pooled instance has an associated *lease* with an expiration time. Clients that hold a stub for the instance periodically renew this lease. When the lease expires the server removes the instance from the pool. The algorithm is similar to the one used in Java RMI distributed garbage collection [9].

On the server side, `LeaseManager` keeps the current expiration time per instance together with a set of client identifiers that currently hold leases on it. The latter is needed because the same instance can be referenced from several clients at once. Each renewal pushes the expiration time forward, and the instance is removed only when none of its clients renews the lease for longer than the configured lease duration plus a small grace period for clock skew. A background coroutine periodically scans the leases and removes expired entries from `RemoteInstancesPool`. The default lease duration is thirty seconds, and the scan runs every ten seconds.

On the client side, tracked stubs are grouped by server URL and renewal requests are batched per URL. Each stub is held through a weak reference, so the client-side garbage collector is free to reclaim it as soon as user code drops it. On every renewal cycle (ten seconds by default, which is below the server lease duration so that a single missed renewal does not reclaim the instance) tracked entries are split into still-reachable and reclaimed, with a renewal request sent for the first group and a release request for the second. The release request lets the server drop the instance immediately if no other client holds it.

This gives reasonable behaviour for the usual failure cases for free. When a client crashes, its lease expires and the instance is removed. When the server restarts, the client receives failed identifiers in the renewal response and notifies user code through the `onLeaseExpired` callback. Transient network errors are absorbed because a missed renewal is retried on the next cycle. Garbage collection is enabled by default but can

be disabled or fine-tuned through `LeaseConfig` and `LeaseRenewalClientConfig`. The leasing layer is transport-agnostic: the core defines the abstract `LeaseClient` interface, and the Ktor integration provides the default implementation with `/lease/renew` and `/lease/release` HTTP routes.

4 Evaluation of Kotlin Remote

To evaluate the framework, four example applications were developed using Kotlin Remote and rewritten using Kotlin RPC. All four applications follow the shared-codebase model: client and server parts live in the same Kotlin project and share data types. This is the only setting Kotlin Remote is designed for, so the evaluation does not attempt to cover scenarios outside of it (for example, separate codebases or cross-language communication). All implementations are functionally equivalent; the only difference is in the way network interactions are organized. As we are interested in network code, we focus on it during the comparison. The business logic of the applications (database access, command line user interface, etc.) is roughly the same in all implementations. Source code is available in the `integration-tests` folder of the Kotlin Remote repository [25].

We compare Kotlin Remote only with Kotlin RPC. The reason is that for the Kotlin-only shared-codebase setting that Kotlin Remote targets, gRPC and Kotlin RPC are close in features: both rely on a service interface declaration, both generate stubs and server bases at compile time, and both require implementing the service interface. But Kotlin RPC is more concise and more idiomatic for Kotlin. It uses Kotlin types directly with no separate IDL, serialization is provided by `kotlinx.serialization`, and remote functions are suspend functions. gRPC, in comparison, has a separate proto IDL, generated builder-based types that do not look like normal Kotlin code, and a per remote function overhead that includes proto messages and proto-to-domain mapping. In a shared-codebase setting this proto layer essentially duplicates domain types that are already shared. We did prototype gRPC versions for the smaller applications, and the per function overhead in gRPC was always the largest of the three frameworks. So we drop gRPC from the main comparison and focus on the closer alternative. This is not a statement about gRPC in general: it is expected that gRPC pays this overhead in a shared-codebase setting because the setting is not what it is designed for.

Table 1 summarizes how the three frameworks compare on the features that matter for choosing between them. gRPC and Kotlin RPC look similar on the API-style rows: both require a service interface declaration, neither marks the remoteness of a function at its call site, neither supports stateful remote objects with a framework-managed lifecycle, and neither expresses hierarchies between services in the type system. They differ on cross-language support and on whether the transport is fixed. Kotlin Remote differs from both alternatives on the API-style rows and on the absence of streaming, which it currently does not support. The "remoteness marked at call site" row deserves an extra remark: the call expression itself (e.g. `multiply(6, 1)`) syntactically looks the same in all three frameworks; what differs in Kotlin Remote is that the call must be lexically enclosed in a matching `context(config) { ... }` block to type-check, so the call site is at least surrounded by an explicit marker, even if the call itself is not.

Four applications were chosen so that they cover different scenarios. *Todo* is a simple CRUD application with four operations that baselines the per-application setup cost. *Rooms chat* is a chat with rooms that uses remote classes and a peer-to-peer architecture.

Feature	gRPC	Kotlin RPC	Kotlin Remote
Service interface required	Yes (IDL)	Yes (@Rpc)	No
Top level functions	No	No	Yes
Remoteness marked at call site	No	No	enclosing context
Hierarchical context tiers	No	No	Yes
Stateful remote objects	No	No	Yes
Streaming	Yes	Yes (Flow)	No
Kotlin Multiplatform support	Limited	Yes	Yes
Cross-language	Yes	No	No
Transport	HTTP/2 (fixed)	Pluggable (Ktor)	Pluggable (Ktor)

Table 1: Feature comparison of three RPC frameworks for Kotlin.

Social platform is a backend split into ten microservices with thirty-six remote operations, designed to stress per-microservice overhead and orchestration. *CMS* is a small content management system with four hierarchical context tiers and Ktor authentication, designed to exercise the hierarchical contexts feature of Kotlin Remote.

4.1 Example applications

4.1.1 Todo application

This is the smallest of the four applications. It allows the user to create, update, delete and list todo items. State is persisted in an H2 database accessed via Exposed ORM. The server exposes only four remote functions, and these functions are top level functions; they are not methods of any class. Listing 23 shows the entire remote API of the Todo application written using Kotlin Remote.

```

@Remote
context(_: RemoteContext<ServerConfig>)
suspend fun createTodo(request: CreateTodoRequest): Todo =
    Dependencies.repository.create(request)

@Remote
context(_: RemoteContext<ServerConfig>)
suspend fun updateTodo(id: Long, request: UpdateTodoRequest): Todo =
    Dependencies.repository.update(id, request)

@Remote
context(_: RemoteContext<ServerConfig>)
suspend fun deleteTodo(id: Long) = Dependencies.repository.delete(id)

@Remote
context(_: RemoteContext<ServerConfig>)
suspend fun todos(): List<Todo> = Dependencies.repository.readAll()

```

Listing 23: Todo application remote API in Kotlin Remote

The client invokes these functions inside a `context(ServerConfig.asContext()) { ... }`

block, and the framework takes care of the network call. There is also a desktop client built with Compose Multiplatform on top of the same API. From the UI code's point of view, remote calls are suspend functions that have to be invoked from inside a context block.

4.1.2 Rooms chat

The second application is a chat in which users can create rooms, enter and exit them, send messages and list available rooms. The application uses the remote classes feature: a `Room` is a remote class that lives on the central server, while users are represented as remote configurations pointing to user machines. Listing 24 shows the most important parts of the chat code.

```
@RemoteSerializable
class Room private constructor() {
    private val clients = mutableListOf<User>()

    @Remote
    context(_: RemoteContext<RemoteConfig>)
    suspend fun enter(userName: String) { /* add user */ }

    @Remote
    context(_: RemoteContext<RemoteConfig>)
    suspend fun broadcast(fromUserName: String, message: String) {
        clients.filter { it.name != fromUserName }.forEach {
            context(it.remoteContext) { send("$fromUserName: $message") }
        }
    }

    companion object {
        @Remote
        context(_: RemoteContext<RemoteConfig>)
        suspend operator fun invoke(name: String): Room { ... }
    }
}

@Remote
context(_: RemoteContext<RemoteConfig>)
suspend fun send(message: String) { println(message) }
```

Listing 24: Rooms chat: Room class and send function

The interesting part is the body of `broadcast`. Each user has its own `remoteContext` pointing to its machine. Inside `broadcast` the central server switches context to each user in turn and calls the `send` function. This results in a remote call from the central server to each user machine. So in the rooms chat application messages are delivered using genuine peer-to-peer remote calls instead of streaming. Each user runs an embedded remote server on its machine in addition to the central server. This kind of architecture is hard to express using more traditional RPC frameworks, because they assume a strict client-server relationship. In Kotlin Remote any participant can be both client and server, because the remote configuration is a value that can be passed around.

4.1.3 Social platform

A small social network backend split into eight microservices: users, posts, comments, likes, follows, feed, notifications and search. On top of these, two Backend-for-Frontend microservices are added: `webBff` returns rich aggregated objects for a desktop browser, and `mobileBff` returns slim paginated objects for a mobile application. The total is ten microservices and thirty-six remote operations. The first six microservices keep their own state in memory; the other four are pure orchestrators that aggregate data from other microservices. The application was deliberately designed to stress per-microservice overhead and orchestration. Dependencies of every microservice are wired with Koin [12], with one Koin module per microservice.

All ten services use no remote classes. Every remote operation is a top level function annotated with `@Remote`, and all thirty-six of them live in a single `API.kt` file. Listing 25 shows the `getFeed` function from the feed microservice, which uses five other microservices to assemble a feed.

```
@Remote context(_: RemoteContext<FeedServiceConfig>)
suspend fun getFeed(userId: Long): List<FeedItem> {
    val followees = context(FollowServiceConfig.asContext()) {
        getFollowing(userId)
    }
    val posts = followees.flatMap {
        context(PostServiceConfig.asContext()) { listUserPosts(it) }
    }
    return posts.map { p ->
        val author = context(UserServiceConfig.asContext()) {
            getUser(p.authorId)
        }
        val likes = context(LikeServiceConfig.asContext()) {
            getPostLikes(p.id)
        }.size
        val comments = context(CommentServiceConfig.asContext()) {
            getPostComments(p.id)
        }.size
        FeedItem(p, author, likes, comments)
    }
}
```

Listing 25: Service-to-service orchestration in social platform

To call a function from another microservice it is enough to wrap the call into a context block: `context(XxxServiceConfig.asContext()) { ... }`. There are no service stubs to inject, no constructor parameters to thread through, and no startup ordering between microservices. BFF endpoints have the same shape as `getFeed`: switch context to each downstream microservice that is needed and assemble the response. In the Kotlin RPC version of the application, `getFeed` has to be a method of the `FeedService @Rpc` interface, implemented by the `FeedServiceImpl` class that takes five service stubs as constructor parameters (one per downstream microservice). The launcher of the feed microservice has to create those five stubs explicitly before starting the server. Each BFF microservice in Kotlin RPC pays the same cost: an `@Rpc` interface, an implementation class with four or

five stub constructor parameters, and a launcher that builds the stubs in the right order.

The client side benefits from context parameters as well. Listing 26 shows the test function that exercises the platform. It declares context parameters for all ten services in a single `context(...)` block on its signature (some of the declarations are omitted). Inside the function every remote call is a plain unqualified function call, with no per-call context block and no service stub. In the Kotlin RPC version the same function takes ten service stubs as parameters and every call goes through the corresponding stub.

```
context(  
    _: RemoteContext<UserServiceConfig>,  
    _: RemoteContext<PostServiceConfig>,  
    _: RemoteContext<FollowServiceConfig>,  
    _: RemoteContext<LikeServiceConfig>,  
    ...  
)  
suspend fun runTestClient() {  
    val alice = register("alice", "pw", "Alice")  
    val p1 = createPost(CreatePostDto(alice.id, "Hello #world"))  
    likePost(p1.id, alice.id)  
    val feed = getFeed(alice.id)  
    val webHome = getWebHomePage(alice.id)  
    val mobileHome = getMobileHomePage(alice.id, 0, 10)  
}
```

Listing 26: Multi-context client function in social platform

4.1.4 CMS

A small content management system designed specifically to exercise hierarchical contexts, the feature mentioned in section 3.2.1 but not otherwise tested in the previous applications. The CMS exposes fourteen remote operations split into four tiers of capabilities: Guest (anonymous read access), Author (create and edit own articles and comments), Moderator (delete any comment, pin an article) and Admin (promote users, view statistics, purge a user). In Kotlin Remote the four tiers are expressed as a chain of subtyped configs:

```
interface GuestConfig : RemoteConfig  
interface AuthorConfig : GuestConfig  
interface ModeratorConfig : AuthorConfig  
interface AdminConfig : ModeratorConfig
```

Listing 27: Four-tier context hierarchy in CMS

Each remote function declares the minimum tier it requires as its context parameter. Because of covariance and subtyping, any function defined with a lower-tier requirement can be called from a higher-tier context. So a moderator session naturally calls guest-tier functions, and an admin session naturally calls functions of all three lower tiers. This propagation is in the type system; no extra wiring is needed at the call site.

Runtime protection of tiers is done with Ktor basic authentication. The server installs four basic auth providers, one per tier, each accepting credentials only if the authenticated user

has at least the required role. Each tier is exposed under its own protected HTTP route (/call/guest, /call/author, /call/moderator, /call/admin). Each user session class points at the route matching its role and uses the user's credentials.

Hierarchical contexts show their value in cross-tier internal composition. Listing 28 shows the admin-tier `purgeUser` function, which transitively calls moderator-tier and guest-tier functions to remove all articles and comments of a banned user. The helper `dep<T>()` in the listing is a thin wrapper over Koin [12] that resolves a repository instance from the server's dependency injection container; it is unrelated to Kotlin Remote itself and is shown only so that the function bodies are realistic.

```
@Remote context(_: RemoteContext<ModeratorConfig>)
suspend fun removeArticleAndComments(id: Long) {
    val comments = getComments(id) // requires GuestConfig
    comments.forEach {
        deleteAnyComment(it.id) // requires ModeratorConfig
    }
    dep<ArticleRepository>().deleteAny(id)
}

@Remote context(_: RemoteContext<AdminConfig>)
suspend fun purgeUser(login: String): Int {
    val articles = dep<ArticleRepository>().byAuthor(login)
    articles.forEach {
        removeArticleAndComments(it.id) // requires ModeratorConfig
    }
    val droppedComments = dep<CommentRepository>().deleteByAuthor(login)
    promoteUser(login, Role.GUEST) // requires AdminConfig
    return articles.size + droppedComments
}
```

Listing 28: Cross-tier internal composition in CMS

All inner calls in `purgeUser` are remote functions of lower tiers, but on the server they execute locally with no network round trip. When `purgeUser` runs on the server the context provided by the framework is `LocalContext`, which is a subtype of `RemoteContext<X>` for any `X`, so the inner calls dispatch their local bodies. Kotlin RPC does not support hierarchy between `@Rpc` interfaces, so the equivalent project models the four tiers as four independent `@Rpc` interfaces with four implementation classes and four registered routes. Two consequences follow. First, the client has to obtain a separate stub for every tier it wants to call: a moderator client that needs to call a guest-tier function needs both a moderator stub and a guest stub. Second, cross-tier internal calls on the server cannot pass through the service abstraction without paying for a network round trip, so the Kotlin RPC version goes directly to the repositories from within the moderator and admin implementations, with an explanatory comment. This works but it bypasses the service layer.

4.2 Boilerplate comparison with Kotlin RPC

Table 2 shows the size of framework-specific code for each application. By framework-specific code we mean code that exists only because the project uses this particular

RPC framework's abstractions: remote function annotations and signatures (signatures count because they determine serialization), framework configuration classes and objects, `registerService` on the server and `withService` on the client (the analogs of context object definition and context block opening in Kotlin Remote), context blocks and remote calls at their call sites, implementation classes that wrap `@Rpc` interfaces. The following code is excluded:

1. Pure business logic (function bodies, repositories, data classes, CLI text)
2. Ktor transport setup: `HttpClient` and `embeddedServer`, installation of generic Ktor plugins, routing wrappers, JSON serializer configuration
3. Framework-specific transport-binding helpers: `install(KRemote)`, `install(KRpc)`, `rpc("/api")`, `remote("/call")`, `rpcConfig`, `installKRpc()`
4. Imports, package declarations and empty lines

Counts are produced by a deterministic line-report pass that we added to the Kotlin Remote compiler plugin¹. The pass runs as an extra `IrGenerationExtension` that is registered only when the user passes a `lineReport` option to the plugin; in that mode the regular code-generating extensions are skipped, so the same plugin can also be applied to Kotlin RPC modules to measure them under identical rules. The pass walks the IR after frontend resolution, identifies declarations and call sites by fully qualified name (`kotlinx.remote.Remote`, `kotlinx.rpc.annotations.Rpc`, `kotlinx.remote.asContext`, `kotlin.context`, `kotlinx.rpc.withService` and so on), and emits one JSON report per Kotlin module listing every framework-specific source line together with the category that tagged it. Lines tagged by more than one category (e.g. an `@Remote` annotation that lives on the same line as a context parameter) are deduplicated, so the totals shown below count each physical line at most once. A single Gradle task, `./gradlew lineReports`, from inside `integration-tests` directory compiles every measured module with the pass enabled and prints the side-by-side comparison; the underlying JSON files contain the exact line numbers in each `.kt` file that contributed to the total, so the count is fully auditable.

Application	Kotlin Remote	Kotlin RPC	Per-op (KR / krpc)
Todo (4 ops)	25	25	~ 6 / 6
Social platform (36 remote ops)	150	265	~ 4 / 7
CMS (14 remote ops)	72	103	~ 5 / 7

Table 2: Framework-specific code per application. Rooms chat is excluded; see discussion below.

Three patterns emerge from the table. For the Todo application the two frameworks produce exactly the same number of framework-specific lines (25 versus 25). The reason is that there is simply too little network code in the application for the structural differences between Kotlin Remote and Kotlin RPC to register as a gap. Todo exposes four remote functions consumed from one desktop UI through one server, and on each side the costs cancel almost line for line: Kotlin Remote pays around three lines per function (the `@Remote` annotation, the `context(...)` declaration and the suspend function signature) plus a small `RemoteConfig` object, while Kotlin RPC pays roughly two lines per function spread across

¹Available on the `line-counting-test` branch of the Kotlin Remote repository [25].

an `@Rpc` interface and a matching implementation class plus a few lines of fixed scaffolding for those two types. With only four remote operations there is not enough material for either pattern to compound. The UI side of the comparison is identical: both clients open one service handle at startup and make nine direct remote calls from inside their UI handlers. We therefore read this as a baseline result, not a win or a loss for either framework: very small APIs are too small to choose a framework over.

The CMS application has four tiers and fourteen remote operations. Kotlin Remote uses about 30% fewer framework-specific lines than Kotlin RPC (72 versus 103), and the difference is concentrated in the implementation classes: Kotlin RPC requires four `@Rpc` interface declarations plus four implementation classes whose bodies do nothing beyond delegating to repositories, while Kotlin Remote expresses the same information as fourteen top-level `@Remote` functions and four short `RemoteConfig` subtypes. Hierarchical contexts also pay off here: the Kotlin RPC version has to obtain a separate stub for each tier the client wants to call, while in Kotlin Remote any session that satisfies a higher-tier configuration also satisfies all lower tiers via subtyping. Cross-tier internal composition is local in Kotlin Remote (calls dispatch their local bodies because `LocalContext` is a subtype of `RemoteContext<X>` for any `X`), and either makes spurious network calls back to localhost or has to step around the service abstraction in Kotlin RPC.

The social platform shows the largest absolute gap (150 versus 265, Kotlin Remote is about 43% shorter). Per operation that is roughly four lines for Kotlin Remote and seven for Kotlin RPC. Two things contribute. First, ten microservices in Kotlin RPC mean ten implementation classes that mostly delegate to repositories. Second, orchestrating microservices like `feed` and the BFFs require explicit stub creation (`withService` calls) for every dependent microservice, and matching `registerService` calls inside `AllLauncher`. Kotlin Remote replaces both with one `API.kt` file of top-level functions plus one `Configs.kt` with ten `RemoteConfig` objects.

Rooms chat is omitted from Table 2 because the two implementations do different things at the architectural level. In Kotlin Remote each user runs its own embedded server and message delivery is done as a remote call from the central server to the user machine using stored `RemoteContext` values. In Kotlin RPC the central server keeps a per-user mailbox and pushes messages over a server-side stream (`Flow<ChatMessage>`). The Kotlin Remote version needs two `RemoteConfig` classes (one per machine role), nine `@Remote` functions and a `@RemoteSerializable` Room class; the Kotlin RPC version expresses the same behaviour with a single `@Rpc` interface plus its server-side stream subscription. Comparing the two on a single line-count axis would conflate Kotlin Remote's lack of streaming support with the real boilerplate question, so we discuss rooms chat qualitatively rather than place a number on it.

4.2.1 Per-function boilerplate overhead

Another way to compare the frameworks is to count how much code is needed to add a new remote function to an existing application. In Kotlin Remote it is two extra lines: the `@Remote` annotation and the `context(...)` declaration. The function body is the actual implementation. In Kotlin RPC each function has to be added in two places: once as a signature in the service interface and once as an override in the implementation class. Listing 29 shows the comparison.

```

// Kotlin Remote: 3 lines, no other file has to be modified
@Remote
context(_: RemoteContext<ServerConfig>)
suspend fun createTodo(request: CreateTodoRequest): Todo =
    repository.create(request)

// Kotlin RPC: signature in @Rpc interface + override in impl class
interface TodoService {
    suspend fun createTodo(request: CreateTodoRequest): Todo
}
class TodoServiceImpl(...) : TodoService {
    override suspend fun createTodo(request: CreateTodoRequest): Todo =
        repository.create(request)
}

```

Listing 29: Code required for one remote function

4.3 Performance comparison with Kotlin RPC

Previous subsections measured user code. Here we measure what the user pays at runtime and in compiled artifact for the same boilerplate reduction. We ran four benchmarks against Kotlin RPC and a plain Kotlin baseline. Source code is in directories `integration-tests/bench` and `integration-tests/growth-bench` of the Kotlin Remote repository [25] on branch `benchmarks-tests`. Measurements were done on Intel Core Ultra 9 185H, Ubuntu 24.04, Temurin 21.0.7, Kotlin 2.2.20, Ktor 3.2.1 and Kotlin RPC 0.10.1, with server and client running in the same JVM over loopback. Per-call benchmarks use 2 000 warmup and 10 000 measured calls. We report median (p50) and 99th percentile (p99) latency; p50 reflects typical-case behavior, p99 reflects tail behavior. Microbenchmarks use 200 000 warmup and 1 000 000 measured iterations.

4.3.1 Per-call latency

Both frameworks use Ktor as the transport, but defaults differ. Kotlin Remote sends one HTTP POST per call, Kotlin RPC sends one frame over a persistent WebSocket. Table 3 shows round-trip latency for identical operations.

Operation	KR p50	KR p99	Kotlin RPC p50	Kotlin RPC p99
ping()	438 us	1076 us	351 us	835 us
echoLong(0)	312 us	478 us	298 us	464 us
echoString(1B)	312 us	457 us	196 us	342 us
echoString(1KiB)	327 us	479 us	213 us	365 us
createTodo(req)	314 us	470 us	195 us	351 us
listTodos(100)	393 us	559 us	339 us	498 us

Table 3: Round-trip latency per call, localhost, 10 000 samples. KR = Kotlin Remote.

The gap stays roughly constant up to 1 KiB payload size, so it is dominated by per-request HTTP overhead and not by framework logic. The Kotlin Remote transport is pluggable

through `RemoteClient`, so an HTTP/2 connection-reusing or in-process transport would close most of this gap.

4.3.2 Local dispatch overhead

In section 3.2.1 we showed that a `@Remote` function dispatched in `LocalContext` runs locally without a network call. This happens when the function executes on the server, or when one tier of the CMS hierarchy calls a lower tier internally. Table 4 compares the same function in three configurations.

Case	ns per call
<code>suspend fun mulPlain(a, b)</code>	15.1
<code>@Remote suspend fun mulRemote(a, b) in LocalContext</code>	21.8
<code>@Remote suspend fun mulRemote(a, b) in ConfiguredContext (network)</code>	470 243

Table 4: Cost of dispatching a remote function in each of the three contexts, 1 000 000 iterations.

The overhead in the local case is basically non-existent with one `is`-check on the context parameter and one call dispatch.

4.3.3 Distributed garbage collection

We measured the cost of a single `renewLeases` call on the server, the renewal request body size, and the memory footprint of a leased instance.

N live stubs	renewal time	per id	request body	traffic per minute
1	0.5 us	484 ns	59 B	354 B
100	5.4 us	54 ns	349 B	2.0 KiB
1 000	47.8 us	48 ns	3.9 KiB	23.1 KiB
10 000	412 us	41 ns	48 KiB	287 KiB
100 000	4.8 ms	48 ns	575 KiB	3.4 MiB

Table 5: Server-side `LeaseManager.renewLeases` cost and on-wire size as a function of the number of tracked instances. Default renewal interval is 10 seconds, so each client sends the request six times per minute.

Renewal cost is linear and basically one synchronized hash-map lookup per id. The periodic `cleanupExpiredInstances` sweep is about 310 ns per expired id.

Table 6 shows server-side heap occupied by a growing instance pool, sampled after a forced garbage collection. Per stub cost includes the user object, the entry in `RemoteInstancesPool`, the `LeaseEntry` record and its `clientIds` set.

Per stub cost is constant as the pool grows, so server memory scales linearly with the number of tracked instances.

N live stubs	cumulative heap	per stub
10 000	8.2 MiB	374 B
100 000	42.1 MiB	386 B
500 000	197.9 MiB	399 B

Table 6: Server-side memory footprint as a function of pool size, sampled after forced GC.

4.3.4 Per-function bytecode footprint

Boilerplate reduction from Table 2 measures user code. The complementary question is how much bytecode the compiler plugin emits per remote function. We generated a single source file with N identical `@Remote suspend fun fN(x: Long): Long = x` functions for Kotlin Remote, and N identical methods of one `@Rpc` interface for Kotlin RPC. We compiled each to a JAR for $N \in \{0, 1, 10, 100, 200, 500, 800, 1000\}$ and measured class file sizes. For Kotlin Remote we ran tests with one top level call to `genCallableMap()`.

Configuration	bytes/function
Kotlin Remote	$\sim 2\,190$
Kotlin RPC	$\sim 2\,500$

Table 7: Per-function bytecode cost ($N \leq 500$).

Both frameworks fail to compile at $N = 800$. The failure mode is identical: the static initializer where the per-function metadata is emitted exceeds the JVM 64 KiB cap. In Kotlin Remote this is the top-level `Kt` class initializer with the expanded `genCallableMap()` literal. In Kotlin RPC this is the static initializer of the per-interface stub class. This is not a real limit in practice, because a realistic service splits its API across several modules or several `@Rpc` interfaces.

4.3.5 Discussion

The boilerplate reduction reported earlier in this section is not bought by a runtime or compiled-size penalty. Local dispatch is basically free, distributed garbage collection stays well under a 1% of CPU even at hundreds of thousands of live references, per-function bytecode is within 12% of Kotlin RPC, and per-call latency on the default Ktor HTTP transport is around 100 us above Kotlin RPC’s default WebSocket transport, which is a transport-level rather than framework-level difference.

5 Limitations and Future Work

This section describes the known limitations of Kotlin Remote and outlines directions for future work identified during development. Subsection 5.1 discusses limitations that follow from the framework’s design and current implementation. Subsection 5.2 outlines directions for future work that go beyond addressing those limitations.

5.1 Limitations

5.1.1 Narrow scope of application

Kotlin Remote is optimized to work in a very specific environment: Kotlin only shared-codebase projects. This is the most important limitation of the framework and it is by design. Kotlin Remote does not try to be a general purpose RPC framework. It does not support cross-language communication, it is not meant for projects in which client and server are developed independently in separate codebases, and it is not meant for projects in which client and server have separate release cycles. In all of these scenarios general purpose frameworks like gRPC, Ktor or Kotlin RPC remain the correct choice and Kotlin Remote should not be considered as their replacement.

Several specific consequences follow from this scope decision. First, both client and server have to be written in Kotlin. Second, signatures of remote functions are shared between client and server through the common code of the project, so a change in remote function signature is a compile-time change for both sides at once. There is no notion of versioned API contract that clients can be incompatible with, this is left to the project's own release process. Third, the framework is not concerned with public-facing API ergonomics (stable URL design, custom HTTP headers, status codes), which are important for public REST APIs. Kotlin Remote endpoint is meant to be consumed by Kotlin clients from the same project, not by arbitrary third-party clients.

Within this scope Kotlin Remote gives substantial reduction of boilerplate, as shown in the previous sections. Outside this scope it is simply the wrong tool.

5.1.2 Dependency on experimental Kotlin features

Kotlin Remote uses context parameters, which are an experimental Kotlin feature at the time of writing. The user has to add the `-Xcontext-parameters` compiler flag to enable them, and the API of context parameters can change in future Kotlin releases. Kotlin Remote also has to be updated for every new Kotlin compiler version because the compiler plugin API is not stable.

5.2 Future work

Apart from addressing the limitations described in section [5.1](#), several directions for future work were identified during development.

5.2.1 Code slicing

Currently, every machine that runs a Kotlin Remote application contains the full compiled artifact, including code that is meant to be executed on other machines. For the social platform this means that the user microservice contains compiled code of all other microservices, even though it never executes them. Type information about the target machine is already encoded in remote contexts (`UserServiceConfig`, `PostServiceConfig`, etc.). This information could be used by the compiler plugin to slice the application into per node artifacts and include in each artifact only the code reachable from that node. This is essentially what tierless languages discussed in section [2.7](#) do at the language level. Doing it as a compiler plugin pass over an already typed Kotlin program is a feasible direction.

5.2.2 Remote lambdas

Kotlin Remote currently supports only named functions and methods of remote classes as remote callables. Lambdas (function values) are not supported. Adding remote lambdas would allow higher order remote functions like `remoteList.map { remoteFunction(it) }` where the mapping is done on the remote machine. This would require making function values serializable, which is a hard problem in general but seems tractable for closed-over values that are themselves serializable.

5.2.3 Bidirectional streaming

Adding streaming support, possibly via Kotlin `Flow` similarly to Kotlin RPC, would make Kotlin Remote suitable for applications like chats, push notifications and live updates without the need to set up a peer-to-peer architecture. Bidirectional streaming with backpressure is a natural fit for Kotlin coroutines.

6 Conclusions

In this thesis we developed Kotlin Remote, an RPC framework for shared-codebase Kotlin Multiplatform projects. The framework uses Kotlin's experimental context parameters feature to express remote calls. A context parameter of a remote function carries information about the target machine, which makes the function callable on that machine, and at the same time appears in the function signature, which makes the remoteness of the function explicit at the declaration level and forces every call site to be lexically enclosed in a matching `context(...)` block. This is a departure from the service-oriented RMI model adopted by gRPC and Kotlin RPC, where remoteness is hidden behind service interfaces and at the call site is completely indistinguishable from a local call.

The four objectives stated in section 1.1 were addressed as follows. The first objective, to prototype an RPC framework based on context parameters, is realized by the `core-functions` module and the `@Remote` surface API described in section 3.2. The second objective, to prototype support for distributed objects, is realized by the `core-classes` module: it contains both the `@RemoteSerializable` stub mechanism (section 3.6) and the leased distributed garbage collector (section 3.7). The third objective, to implement the framework as a Kotlin compiler plugin together with a runtime library, is realized by the `compiler-plugin` module (section 3.3) together with the three runtime modules `core-functions`, `core-classes` and `core-ktor` (section 3.1). The fourth objective, to evaluate the result on example projects, is realized by the four applications of section 4.2 together with the deterministic line-report compiler pass.

The evaluation gives a more nuanced answer to the main research question than a single number. On the two larger applications, the social platform with ten microservices (section 4.1.3) and the CMS with four hierarchical capability tiers (section 4.1.4), Kotlin Remote reduces framework-specific code by approximately 43% and 30% respectively compared to Kotlin RPC. On the smallest application, Todo (four remote functions invoked from a desktop UI), the two frameworks tie at the same line count (25 vs 25): the application has too little network code for the structural differences between the two frameworks to compound into a measurable gap. Rooms chat is not directly comparable because the two implementations make architecturally different choices (peer-to-peer remote calls vs server-pushed `Flow`). The honest summary is that Kotlin Remote reduces boilerplate once

the network surface of an application grows beyond a handful of operations on a single service: the difference between paying per remote function (Kotlin Remote) and paying per service interface plus implementation class (Kotlin RPC) only shows up when there are several services or tiers to express. On a tiny API the two frameworks are interchangeable in size; on a project with multiple microservices or hierarchical tiers Kotlin Remote wins. The framework-overhead benchmarks of section 4.3.1 show that this boilerplate reduction does not come at the cost of runtime performance. Local dispatch of a `@Remote` function in `LocalContext` costs 6.7 ns above a plain suspend call (section 4.3.2) and per-function bytecode is within 12% of Kotlin RPC (section 4.3.4). The per-call latency gap on the default Ktor HTTP transport is a transport-level rather than a framework-level cost. The goal of the thesis stated in section 1.1 is therefore reached for the class of applications it was designed for, while we deliberately do not claim that Kotlin Remote is a better choice for every shared-codebase project.

Kotlin Remote has limitations discussed in section 5.1. The most important one is the narrow scope of application described in section 5.1.1. The framework also depends on experimental Kotlin features. Several directions for future work were identified during development, including per node code slicing, remote lambdas and bidirectional streaming.

References

- [1] Andrew D. Birrell and Bruce Jay Nelson. “Implementing remote procedure calls”. In: *ACM Trans. Comput. Syst.* 2.1 (Feb. 1984), pp. 39–59. ISSN: 0734-2071. DOI: [10.1145/2080.357392](https://doi.org/10.1145/2080.357392). URL: <https://dl.acm.org/doi/10.1145/2080.357392> (visited on 02/07/2026).
- [2] Adam Chlipala. “Ur/Web: a simple model for programming the web”. In: *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. POPL ’15. New York, NY, USA: Association for Computing Machinery, Jan. 2015, pp. 153–165. ISBN: 978-1-4503-3300-9. DOI: [10.1145/2676726.2677004](https://doi.org/10.1145/2676726.2677004). URL: <https://doi.org/10.1145/2676726.2677004>.
- [3] *Coeffects: Context-aware programming languages*. URL: <https://tomasz.net/coeffects/> (visited on 10/15/2025).
- [4] Ezra Cooper et al. “Links: web programming without tiers”. In: *Proceedings of the 5th international conference on Formal methods for components and objects*. FMCO’06. Berlin, Heidelberg: Springer-Verlag, 2007, pp. 266–296. ISBN: 978-3-540-74791-8. DOI: [10.1007/978-3-540-74792-5_12](https://doi.org/10.1007/978-3-540-74792-5_12). URL: https://doi.org/10.1007/978-3-540-74792-5_12.
- [5] *Coroutines | Kotlin*. en. URL: <https://kotlinlang.org/docs/coroutines-overview.html> (visited on 04/15/2026).
- [6] Roy Thomas Fielding. “Architectural styles and the design of network-based software architectures”. AAI9980887. PhD thesis. University of California, Irvine, 2000.
- [7] *Gradle Build Tool*. en-US. Feb. 2026. URL: <https://gradle.org/> (visited on 02/13/2026).
- [8] *gRPC*. en. URL: <https://grpc.io/> (visited on 02/07/2026).
- [9] *Java Development Kit Version 17 API Specification*. en. URL: <https://docs.oracle.com/en/java/javase/17/docs/specs/rmi/index.html> (visited on 02/10/2026).
- [10] *KEEP/notes/code-coloring.md at master · Kotlin/KEEP*. URL: <https://github.com/Kotlin/KEEP/blob/master/notes/code-coloring.md> (visited on 10/15/2025).
- [11] *KEEP/proposals/KEEP-0367-context-parameters.md at main · Kotlin/KEEP*. URL: <https://github.com/Kotlin/KEEP/blob/main/proposals/KEEP-0367-context-parameters.md> (visited on 10/15/2025).
- [12] *Koin: a pragmatic lightweight dependency injection framework for Kotlin*. en. URL: <https://insert-koin.io/> (visited on 04/15/2026).
- [13] *Kotlin Multiplatform*. en. URL: <https://kotlinlang.org/docs/multiplatform.html> (visited on 04/15/2026).
- [14] *Kotlin/kotlinx.rpc*. URL: <https://github.com/Kotlin/kotlinx.rpc> (visited on 04/15/2026).
- [15] *Kotlin/kotlinx.serialization*. original-date: 2017-07-20T11:25:23Z. Feb. 2026. URL: <https://github.com/Kotlin/kotlinx.serialization> (visited on 02/11/2026).
- [16] *Ktor: Build Asynchronous Servers and Clients in Kotlin*. en. URL: <https://ktor.io/> (visited on 02/11/2026).
- [17] Daan Leijen. *Koka: a Functional Language with Effects*. URL: <https://koka-lang.github.io/> (visited on 04/15/2026).

- [18] Jeffrey R. Lewis et al. "Implicit parameters: dynamic scoping with static types". In: *Proceedings of the 27th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. POPL '00. New York, NY, USA: Association for Computing Machinery, Jan. 2000, pp. 108–118. ISBN: 978-1-58113-125-3. DOI: [10.1145/325694.325708](https://doi.org/10.1145/325694.325708). URL: <https://doi.org/10.1145/325694.325708> (visited on 04/15/2026).
- [19] Tomas Petricek, Dominic Orchard, and Alan Mycroft. "Coeffects: unified static analysis of context-dependence". In: *Proceedings of the 40th international conference on Automata, Languages, and Programming - Volume Part II*. ICALP'13. Berlin, Heidelberg: Springer-Verlag, July 2013, pp. 385–397. ISBN: 978-3-642-39211-5. DOI: [10.1007/978-3-642-39212-2_35](https://doi.org/10.1007/978-3-642-39212-2_35). URL: https://doi.org/10.1007/978-3-642-39212-2_35 (visited on 02/09/2026).
- [20] František Plášil and Michael Stal. "An architectural view of distributed objects and components in CORBA, Java RMI and COM/DCOM". en. In: *Software - Concepts & Tools* 19.1 (Mar. 1998), pp. 14–28. ISSN: 1432-2188. DOI: [10.1007/s003780050004](https://doi.org/10.1007/s003780050004). URL: <https://doi.org/10.1007/s003780050004> (visited on 02/07/2026).
- [21] Gordon Plotkin and Matija Pretnar. "Handlers of algebraic effects". In: *Proceedings of the 18th European Symposium on Programming Languages and Systems: Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2009*. ESOP '09. Berlin, Heidelberg: Springer-Verlag, Mar. 2009, pp. 80–94. ISBN: 978-3-642-00589-3. DOI: [10.1007/978-3-642-00590-9_7](https://doi.org/10.1007/978-3-642-00590-9_7). URL: https://doi.org/10.1007/978-3-642-00590-9_7.
- [22] *Protocol Buffers*. en. URL: <https://protobuf.dev/> (visited on 04/15/2026).
- [23] Louis Ryan|. *gRPC Motivation and Design Principles*. en. Section: blog. Sept. 2015. URL: <https://grpc.io/blog/principles/> (visited on 02/10/2026).
- [24] *Servant - haskell-servant*. URL: <https://www.servant.dev/> (visited on 02/08/2026).
- [25] Aleksandr Stupnikov. *enotvtapke/kotlin-remote*. original-date: 2025-11-30T22:59:26Z. May 2026. URL: <https://github.com/enotvtapke/kotlin-remote> (visited on 05/18/2026).
- [26] Andrew S Tanenbaum and Robbert van Renesse. "A Critique of the Remote Procedure Call Paradigm". en. In: ().
- [27] Andrew S. Tanenbaum and Maarten van Steen. *Distributed Systems: Principles and Paradigms (2nd Edition)*. USA: Prentice-Hall, Inc., Sept. 2006. ISBN: 978-0-13-239227-3.
- [28] *The Opa Language*. URL: <http://opalang.org/> (visited on 10/15/2025).
- [29] *What Color is Your Function? – journal.stuffwithstuff.com*. URL: <https://journal.stuffwithstuff.com/2015/02/01/what-color-is-your-function/> (visited on 10/15/2025).
- [30] James E. White. "A high-level framework for network-based resource sharing". In: *Proceedings of the June 7-10, 1976, national computer conference and exposition*. AFIPS '76. New York, NY, USA: Association for Computing Machinery, June 1976, pp. 561–570. ISBN: 978-1-4503-7917-5. DOI: [10.1145/1499799.1499878](https://dl.acm.org/doi/10.1145/1499799.1499878). URL: <https://dl.acm.org/doi/10.1145/1499799.1499878> (visited on 02/10/2026).

- [31] Ann Wollrath, Roger Riggs, and Jim Waldo. "A distributed object model for the java™ system". In: *Proceedings of the 2nd conference on USENIX Conference on Object-Oriented Technologies (COOTS) - Volume 2*. COOTS'96. USA: USENIX Association, June 1996, p. 17. (Visited on 02/07/2026).